Decisions, and what each one costs

Every entry could have gone the other way. What makes them decisions rather than defaults is that **the cost is understood and accepted on purpose.** The cost is recorded alongside, because that is the part that evaporates first — and the part a future change needs before it reopens anything here.

DECISION	WHY	WHAT IT COSTS
No backend at all	This is a volunteer-run club of a few dozen people, built and maintained by one person. A server means hosting to maintain, secrets to rotate, and something that can fail at the weekend. A static site has none of that.	No scheduled jobs, no server-side reads, no credentials, no cross-client transactions. Four limitations elsewhere in this system trace directly to this line.
PIN tiers instead of user accounts	Tiered access is needed; maintaining a list of accounts is not acceptable. Three tiers opened by PIN — nothing to sign up for, nothing to reset, nobody to off-board.	Shared PINs mean shared responsibility. The audit log knows which PIN acted, not which person — attribution holds only while people use their own.
Hash PINs, never encrypt them	With no server, any decryption key would ship inside the public JavaScript beside what it protects. Hashing never needs reversing, so there is nothing to leak.	A short PIN still has a small keyspace. Hashing keeps plain PINs out of the database; it does not make a short one resistant to a determined attempt on a stolen hash.
Admin registry in its own document	Admins can already edit the directory. An "is Admin" flag stored there would sit inside the thing it controls, letting any Admin read or grant access, including their own.	One more document to reason about, and one more thing that must be present for access checks to work at all.
Link out to the club calendar	A direct fetch of the calendar feed is blocked by the browser. Rather than fake it, the app links to the real schedule and says plainly that court numbers are unverified.	Court conflicts depend entirely on a human looking. The app will print "Court 3" while a lesson is booked on it.
Store as little as the job needs	A club roster could reasonably hold phone numbers, emails and addresses. None were added — the directory carries only what the pairing engine uses, so there is no contact information to expose or protect. Names and playing times are also kept out of search engines.	Contact details have to live somewhere else, so the app can never become the single place member information is kept.
Ratings stay manual	At a recreational club a rating is a social fact as much as a measured one. Letting last Sunday quietly move someone's number would cause more arguments than it settles.	A rating can sit stale while a player's form moves, and nothing notices. A computed form signal, kept separate from the organizer's number, remains the favoured direction.

Decisions about the data and the people

What the app rates players on, what it keeps about them, and what it shows to a room. **Each one is narrower than the obvious alternative**, and narrower on purpose.

DECISION	WHY	WHAT IT COSTS
CTA leads, USTA falls back	The club rates everyone; USTA covers only those who play league tennis. Rating half the club on an external scale and half on an internal one means the two halves were never calibrated against each other. The club's own judgment of how someone plays here is the more useful number.	A CTA rating that was set carelessly now drives pairing directly, so the directory needs a considered pass before the numbers can be trusted.
Serving scored on the team, not the player	A strong serve is already part of why someone is rated where they are, so folding it into their rating would count it twice. What a rating cannot express is a property of the pair: two weak servers together struggle to hold serve however well they rally.	Another term competing with skill balance for influence. Kept deliberately light so it breaks ties rather than driving decisions.
No age or health notes in the app	Publishing ages, and holding health-adjacent free text behind a single shared PIN, is more personal data than a pairing sheet needs. Both remain in storage and in the export, so nothing is destroyed.	The organizer loses a place to note a knock. Modelling that properly — a limitation flag the engine can act on — is a separate build and not yet done.
Ratings behind a tap, not beside a name	A number printed next to someone's name on a screen anyone can open reads as a public ranking. One tap on the info control shows the same number to whoever actually wants it.	The organizer loses an at-a-glance scan of the whole list and has to tap per player. Ratings are still visible to anyone who looks — this reduces exposure, it does not make them private.
Badges stay rare and positive	A streak flame at three wins is uncommon enough to mean something. Badges that fire most weeks become wallpaper, and anything marking a player out negatively has no place on a social club's sheet.	Every badge is derived from logged results, so incomplete logging distorts them — an unlogged loss does not break a streak. Further badges are held until the logging habit is reliable.

Guarding against silent failure

The defining failure mode of a system with no server and no operator is **something succeeding incorrectly and reporting success.** No error, no crash, no log entry. Four designs exist specifically to prevent that.

"Not found" is not "cannot reach"

The storage layer tags every read failure distinctly. First-time setup runs only on a confirmed missing document, and every PIN check refuses *any* PIN when the registry cannot be confirmed either way.

Without it, a browser unable to reach the database looks identical to a fresh install — and the app would rebuild default access in memory and accept a default PIN.

Import merges field by field

Only fields the spreadsheet actually supplies are written. A blank cell means "nothing to say", never "erase this".

Replacing the whole record instead would silently reset every hand-entered value a names-and-ratings roster does not carry.

Per-browser weekly sheets

Each browser namespaces its own sheet, so collisions become impossible rather than unlikely — with no locking or merge logic to get wrong.

One shared document, rewritten in full on every save, means two organizers with the app open erase each other's work without either of them seeing anything.

A copy reads the source fresh

Taking a copy of another sheet re-reads it at that moment rather than using the cached listing, and says so if it cannot.

Otherwise a sheet adjusted after the list was loaded is copied in its pre-adjustment state: right players, wrong lineup, no sign anything is out of date.

STANDING RULES FOR FUTURE WORK

The directory and match history are irreplaceable

Pairings regenerate in seconds; hand-entered player data and logged results cannot be reconstructed at all. Any change touching either needs explicit preservation tests before it ships, not after.

Confirm before packaging a release

Summarize what is in a build and get an explicit go-ahead before producing files, especially for anything touching access or data.

Prefer a preference to a rule

Court preferences, split-by-sex and singles all degrade quietly when the headcount cannot support them. A session that can be played beats a rule that cannot be honoured.

Delete dead code rather than leaving it

Code that cannot work misleads whoever reads the file next.

Reopening any of this is fine. Reopen the reason as well as the outcome — several entries look like unnecessary complexity until you know what they are load-bearing for.